Final Slips Selection: 1,3,4,5,9,10,11,12,13,16,20,23,25,27,28

SLIP 1

Q.1) Write a C Menu driven Program to implement following functionality
a) Accept Available
b) Display Allocation, Max
c) Display the contents of need matrix
d) Display Available

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 10

int max[MAX][MAX], need[MAX][MAX], allocation[MAX][MAX];
int available[MAX], work[MAX], finish[MAX], request[MAX], safeSeq[MAX];
int np, nr;

void accept_matrix(int A[MAX][MAX])
{
   int i, j;
   for(i = 0; i < np; i++)
      for(j = 0; j < nr; j++)
         scanf("%d", &A[i][j]);
}

void accept_vector(int A[MAX])
{
   int i;
   for(i = 0; i < nr; i++)
      scanf("%d", &A[i]);
}

void calculate_need()
{
   int i, j;
   for(i = 0; i < np; i++)
      for(j = 0; j < nr; j++)
         need[i][j] = max[i][j] - allocation[i][j];
   printf("\nNeed matrix calculated successfully.\n");
}

void display()
{
```

```
    int i, j;
    printf("\nAllocation\tMax\t\tNeed\n");
    for(i = 0; i < np; i++)
    {
        for(j = 0; j < nr; j++)
            printf("%d ", allocation[i][j]);
        printf("\t\t");

        for(j = 0; j < nr; j++)
            printf("%d ", max[i][j]);

        printf("\t\t");

        for(j = 0; j < nr; j++)
            printf("%d ", need[i][j]);
        printf("\n");
    }
    printf("\nAvailable: ");
    for(i = 0; i < nr; i++)
        printf("%d ", available[i]);
    printf("\n");
}

int main()
{
    int choice;
    printf("Enter number of processes: ");
    scanf("%d", &np);
    printf("Enter number of resources: ");
    scanf("%d", &nr);

    do
    {
        printf("\n----- menu-----\n");
        printf("1. Accept Allocation\n");
        printf("2. Accept Max\n");
        printf("3. Calculate Need\n");
        printf("4. Accept Available\n");
        printf("5. Display Matrices\n");
        printf("6. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch(choice)
```

```
        {
            case 1:
                printf("Enter Allocation Matrix:\n");
                accept_matrix(allocation);
                break;
            case 2:
                printf("Enter Max Matrix:\n");
                accept_matrix(max);
                break;
            case 3:
                calculate_need();
                break;
            case 4:
                printf("Enter Available Vector:\n");
                accept_vector(available);
                break;
            case 5:
                display();
                break;
        }
    } while(choice != 6);

    return 0;
}
```

Q.2 Write a C program that behaves like a shell which displays the command prompt '$'. It accepts the command,
tokenize the command line and execute it by creating the child process. Also implement the additional
command 'count' as
a. $ count c filename: It will display the number of characters in given file
b. $ count w filename: It will display the number of words in given file
c. $ count l filename: It will display the number of lines in given file

```
#include <sys/types.h>
#include <sys/wait.h>
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

void make_toks(char *s, char *tok[])
```

```
{
   int i = 0;
   char *p;

   p = strtok(s, " ");
   while(p != NULL)
   {
      tok[i++] = p;
      p = strtok(NULL, " ");
   }
   tok[i] = NULL;
}

void count(char *fn, char op)
{
   int fh, cc = 0, wc = 0, lc = 0;
   char c;

   fh = open(fn, O_RDONLY);
   if(fh == -1)
   {
      printf("File %s not found.\n", fn);
      return;
   }

   while(read(fh, &c, 1) > 0)
   {
      if(c == ' ')
         wc++;
      else if(c == '\n')
      {
         wc++;
         lc++;
      }
      cc++;
   }
   close(fh);

   switch(op)
   {
      case 'c':
         printf("No.of characters:%d\n", cc);
         break;
      case 'w':
```

```
            printf("No.of words:%d\n", wc);
            break;
        case 'l':
            printf("No.of lines:%d\n", lc);
            break;
    }
}

int main()
{
    char buff[80], *args[10];
    int pid;

    while(1)
    {
        printf("myshell$");
        fflush(stdin);
        fgets(buff, 80, stdin);
        buff[strlen(buff)-1] = '\0';

        make_toks(buff, args);

        if(strcmp(args[0], "quit") == 0)
            break;
        else if(strcmp(args[0], "count") == 0)
            count(args[2], args[1][0]);
        else
        {
            pid = fork();
            if(pid > 0)
                wait(NULL);
            else
            {
                if(execvp(args[0], args) == -1)
                    printf("Bad command.\n");
            }
        }
    }
    return 0;
}
```

SLIP 3

Q.1 Write a C program to simulate Banker's algorithm for the purpose of deadlock avoidance. Consider the following
snapshot of system, A, B, C and D is the resource type.
a) Calculate and display the content of need matrix?
b) Is the system in safe state? If display the safe sequence.

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 10

int max[MAX][MAX], need[MAX][MAX], allocation[MAX][MAX];
int available[MAX], work[MAX], finish[MAX], request[MAX], safeSeq[MAX];
int np, nr;

void accept_matrix(int A[MAX][MAX])
{
   int i, j;
   for(i = 0; i < np; i++)
      for(j = 0; j < nr; j++)
         scanf("%d", &A[i][j]);
}

void accept_vector(int A[MAX])
{
   int i;
   for(i = 0; i < nr; i++)
      scanf("%d", &A[i]);
}

void calculate_need()
{
   int i, j;
   for(i = 0; i < np; i++)
      for(j = 0; j < nr; j++)
         need[i][j] = max[i][j] - allocation[i][j];
   printf("\nNeed matrix calculated successfully.\n");
}

void display()
{
   int i, j;
   printf("\nAllocation\tMax\t\tNeed\n");
```

```
    for(i = 0; i < np; i++)
    {
        for(j = 0; j < nr; j++)
            printf("%d ", allocation[i][j]);
        printf("\t\t");

        for(j = 0; j < nr; j++)
            printf("%d ", max[i][j]);
        printf("\t\t");

        for(j = 0; j < nr; j++)
            printf("%d ", need[i][j]);
        printf("\n");
    }

    printf("\nAvailable: ");
    for(i = 0; i < nr; i++)
        printf("%d ", available[i]);
    printf("\n");
}

int safety_algorithm()
{
    int i, j, count = 0;
    for(i = 0; i < nr; i++)
        work[i] = available[i];

    for(i = 0; i < np; i++)
        finish[i] = 0;

    while(count < np)
    {
        int found = 0;
        for(i = 0; i < np; i++)
        {
            if(finish[i] == 0)
            {
                int flag = 0;
                for(j = 0; j < nr; j++)
                {
                    if(need[i][j] > work[j])
                    {
                        flag = 1;
                        break;
```

```
                }
            }

            if(flag == 0)
            {
                for(j = 0; j < nr; j++)
                    work[j] += allocation[i][j];
                safeSeq[count++] = i;
                finish[i] = 1;
                found = 1;
            }
        }
    }
    if(found == 0)
        return 0;
  }
  return 1;
}

void bankers_algorithm()
{
  if(safety_algorithm())
  {
    int i;
    printf("\nSystem is in SAFE state.\nSafe Sequence: ");
    for(i = 0; i < np; i++)
        printf("P%d ", safeSeq[i]);
    printf("\n");
  }
  else
  {
    printf("\nSystem is in UNSAFE state (Deadlock may occur).\n");
  }
}

void resource_request()
{
  int i, pno;
  printf("Enter process number: ");
  scanf("%d", &pno);
  printf("Enter request vector: ");
  accept_vector(request);

  for(i = 0; i < nr; i++)
```

```
    {
        if(request[i] > need[pno][i])
        {
            printf("Error: Request exceeds maximum need.\n");
            return;
        }
    }

    for(i = 0; i < nr; i++)
    {
        if(request[i] > available[i])
        {
            printf("Resources not available. Process must wait.\n");
            return;
        }
    }

    for(i = 0; i < nr; i++)
    {
        available[i] -= request[i];
        allocation[pno][i] += request[i];
        need[pno][i] -= request[i];
    }

    if(safety_algorithm())
    {
        printf("Request granted. System remains SAFE.\n");
    }
    else
    {
        printf("Request cannot be granted. System becomes UNSAFE.\n");
        for(i = 0; i < nr; i++)
        {
            available[i] += request[i];
            allocation[pno][i] -= request[i];
            need[pno][i] += request[i];
        }
    }
}

int main()
{
    int choice;
    printf("Enter number of processes: ");
```

```
scanf("%d", &np);
printf("Enter number of resources: ");
scanf("%d", &nr);

do
{
   printf("\n----- menu-----\n");
   printf("1. Accept Allocation\n");
   printf("2. Accept Max\n");
   printf("3. Calculate Need\n");
   printf("4. Accept Available\n");
   printf("5. Display Matrices\n");
   printf("6. Apply Banker's Algorithm\n");
   printf("7. Resource Request Algorithm\n");
   printf("8. Exit\n");
   printf("Enter your choice: ");
   scanf("%d", &choice);

   switch(choice)
   {
      case 1:
         printf("Enter Allocation Matrix:\n");
         accept_matrix(allocation);
         break;
      case 2:
         printf("Enter Max Matrix:\n");
         accept_matrix(max);
         break;
      case 3:
         calculate_need();
         break;
      case 4:
         printf("Enter Available Vector:\n");
         accept_vector(available);
         break;
      case 5:
         display();
         break;
      case 6:
         bankers_algorithm();
         break;
      case 7:
         resource_request();
         break;
```

```
        }
    } while(choice != 8);

    return 0;
}
```

Q.2 Write a simulation program for disk scheduling using SSTF algorithm. Accept total number of disk blocks, disk request string, and current head position from the user. Display the list of requests in the order in which it is served. Also display the total number of head moments. 98, 183, 37, 122, 14, 124, 65. Start Head Position: 53

```
#include<stdio.h>
#include<stdlib.h>

int front, rear;

void init()
{
    front = rear = -1;
}

void display(int *Q)
{
    int i;
    printf("\nFront %d\t", front);
    for(i = front; i <= rear; i++)
        printf("\t%d", Q[i]);
}

void enqueu(int *Q, int n, int var)
{
    if(rear == n)
        printf("\nQueue is Full");
    else
    {
        if(front == -1 && rear == -1)
        {
            front++;
            rear++;
        }
        else
            rear++;

        Q[rear] = var;
```

```
    }
}

int SSTF(int *Q, int n, int initial)
{
    int min, diff, index, TotalHeadMoment = 0, count = 0, i;
    while(count != n)
    {
        min = 1000;
        for(i = 0; i < n; i++)
        {
            diff = abs(Q[i] - initial);
            if(min > diff)
            {
                min = diff;
                index = i;
            }
        }
        printf("\nIndex=%d\n", index);
        printf("Disk head moves from %d to %d with seek %d\n", initial, Q[index], min);

        TotalHeadMoment = TotalHeadMoment + min;
        initial = Q[index];
        Q[index] = 1000;
        count++;
```

SLIP 4

Q.1 Implement the Menu driven Banker's algorithm for accepting Allocation, Max from user
a) Accept Available
b) Display Allocation, Max
c) Find Need and display It,
d) Display Available
Consider the system with 3 resources types A, B, and C with 7, 2, 6 instances respectively consider the following snapshot:

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 10

int max[MAX][MAX], need[MAX][MAX], allocation[MAX][MAX];
int available[MAX], work[MAX], finish[MAX], request[MAX], safeSeq[MAX];
int np, nr;
```

```
void accept_matrix(int A[MAX][MAX])
{
    int i, j;
    for(i = 0; i < np; i++)
        for(j = 0; j < nr; j++)
            scanf("%d", &A[i][j]);
}

void accept_vector(int A[MAX])
{
    int i;
    for(i = 0; i < nr; i++)
        scanf("%d", &A[i]);
}

void calculate_need()
{
    int i, j;
    for(i = 0; i < np; i++)
        for(j = 0; j < nr; j++)
            need[i][j] = max[i][j] - allocation[i][j];
    printf("\nNeed matrix calculated successfully.\n");
}

void display()
{
    int i, j;
    printf("\nAllocation\tMax\t\tNeed\n");
    for(i = 0; i < np; i++)
    {
        for(j = 0; j < nr; j++)
            printf("%d ", allocation[i][j]);
        printf("\t\t");

        for(j = 0; j < nr; j++)
            printf("%d ", max[i][j]);
        printf("\t\t");

        for(j = 0; j < nr; j++)
            printf("%d ", need[i][j]);
        printf("\n");
    }
    printf("\nAvailable: ");
    for(i = 0; i < nr; i++)
```

```
        printf("%d ", available[i]);
    printf("\n");
}

int main()
{
    int choice;
    printf("Enter number of processes: ");
    scanf("%d", &np);
    printf("Enter number of resources: ");
    scanf("%d", &nr);

    do
    {
        printf("\n----- MENU -----\n");
        printf("1. Accept Allocation\n");
        printf("2. Accept Max\n");
        printf("3. Calculate Need\n");
        printf("4. Accept Available\n");
        printf("5. Display Matrices\n");
        printf("6. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch(choice)
        {
            case 1:
                printf("Enter Allocation Matrix:\n");
                accept_matrix(allocation);
                break;
            case 2:
                printf("Enter Max Matrix:\n");
                accept_matrix(max);
                break;
            case 3:
                calculate_need();
                break;
            case 4:
                printf("Enter Available Vector:\n");
                accept_vector(available);
                break;
            case 5:
                display();
                break;
```

```
        }
    } while(choice != 6);

    return 0;
}
```

Q.2 Write a simulation program for disk scheduling using SCAN algorithm. Accept total Number of disk blocks, disk request string, and current head position from the user. Display the list of requests in the order in which it is served. Also display the total number of head moments. 82, 170, 43, 140, 24, 16, 190, 65. Starting Head position= 50 Direction: Left

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int RQ[100], i, j, n, TotalHeadMoment = 0;
    int initial, size, move, temp, index;

    printf("Enter total disk size\n");
    scanf("%d", &size);

    printf("Enter the number of Requests\n");
    scanf("%d", &n);

    printf("Enter the Requests sequence\n");
    for(i = 0; i < n; i++)
        scanf("%d", &RQ[i]);

    printf("Enter initial head position\n");
    scanf("%d", &initial);

    printf("Enter the head movement direction for high 1 and for low 0\n");
    scanf("%d", &move);

    for(i = 0; i < n; i++)
    {
        for(j = 0; j < n - i - 1; j++)
        {
            if(RQ[j] > RQ[j+1])
            {
                temp = RQ[j];
                RQ[j] = RQ[j+1];
```

```
            RQ[j+1] = temp;
        }
    }
}

for(i = 0; i < n; i++)
{
    if(initial < RQ[i])
    {
        index = i;
        break;
    }
}

if(move == 1)
{
    for(i = index; i < n; i++)
    {
        TotalHeadMoment = TotalHeadMoment + abs(RQ[i] - initial);
        initial = RQ[i];
    }

    TotalHeadMoment = TotalHeadMoment + abs(size - RQ[i-1] - 1);
    initial = size - 1;

    for(i = index - 1; i >= 0; i--)
    {
        TotalHeadMoment = TotalHeadMoment + abs(RQ[i] - initial);
        initial = RQ[i];
    }
}

else
{
    for(i = index - 1; i >= 0; i--)
    {
        TotalHeadMoment = TotalHeadMoment + abs(RQ[i] - initial);
        initial = RQ[i];
    }

    TotalHeadMoment = TotalHeadMoment + abs(RQ[i+1] - 0);
    initial = 0;

    for(i = index; i < n; i++)
```

```
        {
            TotalHeadMoment = TotalHeadMoment + abs(RQ[i] - initial);
            initial = RQ[i];
        }
    }

    printf("Total head movement is %d", TotalHeadMoment);

    return 0;
}
```

SLIP 5

Q.1 Write the simulation program for demand paging and show the page scheduling total and number of page faults according to the FIFO page replacement algorithm. Assume the memory of n frames. Reference String: 3,4,5,6,3,4,7,3,4,5,6,7,2,4,6

```
#include <stdio.h>

int main()
{
    int n, i, j, faults = 0, sp = 0, found;
    int ref[] = {3, 4, 5, 6, 3, 4, 7, 3, 4, 5, 6, 7, 2, 4, 6};
    int m = sizeof(ref) / sizeof(ref[0]);
    int frames[20];

    printf("Enter number of frames: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++)
    {
        frames[i] = -1;
    }

    printf("\nReference String: ");
    for(i = 0; i < m; i++)
    {
        printf("%d ", ref[i]);
    }
    printf("\n");

    printf("\nPage Scheduling (FIFO):\n");
    for(i = 0; i < m; i++)
    {
```

```
        found = 0;

        for(j = 0; j < n; j++)
        {
            if(frames[j] == ref[i])
            {
                found = 1;
                break;
            }
        }

        if(!found)
        {
            frames[sp] = ref[i];
            sp = (sp + 1) % n;
            faults++;
        }

        printf("Page %d -> ", ref[i]);
        for(j = 0; j < n; j++)
        {
            if(frames[j] != -1)
                printf("%d ", frames[j]);
            else
                printf("- ");
        }

        if(!found)
            printf("\t(Fault)");

        printf("\n");
    }

    printf("\nTotal Page Faults = %d\n", faults);

    return 0;
}
```

Q2. Consider a system with ‘n’ processes and ‘m’ resource types. Accept number of instances for every resource type. For each process accept the allocation and maximum requirement matrices. Write a program to display the contents of need matrix and to check if the given request of a process can be granted immediately or not. (Use resource request algorithm)

```
#include <stdio.h>
#define MAX 10

int max[MAX][MAX], need[MAX][MAX], allocation[MAX][MAX];
int available[MAX], total_instances[MAX], work[MAX], finish[MAX], request[MAX];
int np, nr;

void accept_matrices()
{
    int i, j;

    printf("\nEnter Total Instances of each resource:\n");
    for(i = 0; i < nr; i++)
    {
        printf("Resource %d: ", i);
        scanf("%d", &total_instances[i]);
    }

    printf("\nEnter Allocation Matrix:\n");
    for(i = 0; i < np; i++)
        for(j = 0; j < nr; j++)
            scanf("%d", &allocation[i][j]);

    printf("\nEnter Max Matrix:\n");
    for(i = 0; i < np; i++)
        for(j = 0; j < nr; j++)
            scanf("%d", &max[i][j]);
}

void calculate_need_and_available()
{
    int i, j, sum;

    for(i = 0; i < np; i++)
        for(j = 0; j < nr; j++)
            need[i][j] = max[i][j] - allocation[i][j];

    for(j = 0; j < nr; j++)
    {
        sum = 0;
        for(i = 0; i < np; i++)
            sum += allocation[i][j];

        available[j] = total_instances[j] - sum;
```

```
    }
}

void display_need()
{
    int i, j;
    printf("\nNeed Matrix:\n");
    for(i = 0; i < np; i++)
    {
        for(j = 0; j < nr; j++)
            printf("%d ", need[i][j]);
        printf("\n");
    }

    printf("\nCurrent Available Resources: ");
    for(i = 0; i < nr; i++)
        printf("%d ", available[i]);
    printf("\n");
}

int safety_algorithm()
{
    int i, j, count = 0;

    for(i = 0; i < nr; i++)
        work[i] = available[i];

    for(i = 0; i < np; i++)
        finish[i] = 0;

    while(count < np)
    {
        int found = 0;
        for(i = 0; i < np; i++)
        {
            if(finish[i] == 0)
            {
                int flag = 0;
                for(j = 0; j < nr; j++)
                {
                    if(need[i][j] > work[j])
                    {
                        flag = 1;
                        break;
```

```
                }
            }

            if(flag == 0)
            {
                for(j = 0; j < nr; j++)
                    work[j] += allocation[i][j];

                finish[i] = 1;
                found = 1;
                count++;
            }
        }
    }
    if(found == 0)
        return 0;
  }
  return 1;
}

void resource_request()
{
  int i, pno;

  printf("\nEnter Process Number for Request (0 to %d): ", np - 1);
  scanf("%d", &pno);

  printf("Enter Request Vector:\n");
  for(i = 0; i < nr; i++)
    scanf("%d", &request[i]);

  for(i = 0; i < nr; i++)
  {
    if(request[i] > need[pno][i])
    {
      printf("\nError: Request exceeds maximum need.\n");
      return;
    }
  }

  for(i = 0; i < nr; i++)
  {
    if(request[i] > available[i])
    {
```

```
            printf("\nResources not available. Process must wait.\n");
            return;
        }
    }

    for(i = 0; i < nr; i++)
    {
        available[i] -= request[i];
        allocation[pno][i] += request[i];
        need[pno][i] -= request[i];
    }

    if(safety_algorithm())
    {
        printf("\nRequest can be granted immediately. System remains in SAFE state.\n");
    }
    else
    {
        printf("\nRequest cannot be granted immediately. System becomes UNSAFE.\n");
        for(i = 0; i < nr; i++)
        {
            available[i] += request[i];
            allocation[pno][i] -= request[i];
            need[pno][i] += request[i];
        }
    }
}

int main()
{
    printf("Enter number of processes: ");
    scanf("%d", &np);

    printf("Enter number of resources: ");
    scanf("%d", &nr);

    accept_matrices();
    calculate_need_and_available();
    display_need();
    resource_request();

    return 0;
}
```

SLIP 9

Q.1 Write the simulation program for demand paging and show the page scheduling and total number of page faults according to the optimal page replacement algorithm. Assume the memory of 'n' frames. Reference String: 8, 5, 7, 8, 5, 7, 2, 3, 7, 3, 5, 9, 4, 6, 2

```
#include<stdio.h>

int main()
{
   int no_of_frames, no_of_pages, frames[10], pages[30], temp[10];
   int found, freeSlot, neverUsed, i, j, k, pos, max, faults = 0;

   printf("Enter number of frames: ");
   scanf("%d", &no_of_frames);

   printf("Enter number of pages: ");
   scanf("%d", &no_of_pages);

   printf("Enter page reference string: ");
   for(i = 0; i < no_of_pages; ++i){
      scanf("%d", &pages[i]);
   }

   for(i = 0; i < no_of_frames; ++i){
      frames[i] = -1;
   }

   printf("\nPage Replacement Process:\n");
   for(i = 0; i < no_of_pages; ++i){
      found = freeSlot = 0;

      for(j = 0; j < no_of_frames; ++j){
         if(frames[j] == pages[i]){
            found = 1;
            break;
         }
      }

      if(!found){
         for(j = 0; j < no_of_frames; ++j){
            if(frames[j] == -1){
               frames[j] = pages[i];
               faults++;
```

```
            freeSlot = 1;
            break;
         }
      }
   }

   if(!found && !freeSlot){
      for(j = 0; j < no_of_frames; ++j){
         temp[j] = -1;
         for(k = i+1; k < no_of_pages; ++k){
            if(frames[j] == pages[k]){
               temp[j] = k;
               break;
            }
         }
      }

      neverUsed = 0;
      for(j = 0; j < no_of_frames; ++j){
         if(temp[j] == -1){
            pos = j;
            neverUsed = 1;
            break;
         }
      }

      if(!neverUsed){
         max = temp[0];
         pos = 0;
         for(j = 1; j < no_of_frames; ++j){
            if(temp[j] >
```

Q.2 Write the program to simulate Non-preemptive Shortest Job First (SJF) scheduling. the arrival time and first CPU burst for different n number of processes should be input to the algorithm. The next CPU burst should be generated randomly. The output should give Gantt chart, turnaround time and waiting time for each process. Also find the average waiting time and turnaround time.

```
#include<stdio.h>

int findWaitingTime(int bt[], int wt[], int n)
{
   int wtavg = 0, i;
```

```
    wt[0] = 0;
    for(i = 1; i < n; i++)
    {
        wt[i] = bt[i-1] + wt[i-1] + 2;
        wtavg = wtavg + wt[i];
    }
    return(wtavg);
}

int findTurnAroundTime(int bt[], int wt[], int tat[], int n)
{
    int tatavg = 0, i;
    for(i = 0; i < n; i++)
    {
        tat[i] = bt[i] + wt[i];
        tatavg = tatavg + tat[i];
    }
    return(tatavg);
}

void sort(int n, int bt[], int p[])
{
    int i, j, pos, temp;
    for(i = 0; i < n; i++)
    {
        pos = i;
        for(j = i + 1; j < n; j++)
        {
            if(bt[j] < bt[pos])
            {
                pos = j;
            }
        }
        temp = at[i];
        at[i] = at[pos];
        at[pos] = temp;

        temp = bt[i];
        bt[i] = bt[pos];
        bt[pos] = temp;

        temp = p[i];
        p[i] = p[pos];
        p[pos] = temp;
```

```
    }
}

void main()
{
    int at[10], bt[10], wt[10], tat[10], p[10], i, n;
    int avgwt, avgtat;

    printf("\nEnter The Number of Process\n");
    scanf("%d", &n);

    printf("\nEnter The Arrival and Burst Time\n");
    for(i = 0; i < n; i++)
    {
        p[i] = i;
        printf("\nProcess %d:\t", p[i]);
        scanf("%d%d", &at[i], &bt[i]);
        printf("\n");
    }

    printf("\nBefore Sorting\n");
    printf("P|\tBT|\n");
    for(i = 0; i < n; i++)
    {
        printf("%d", p[i]);
        printf("\t%d\n", bt[i]);
    }

    sort(n, bt, p);

    printf("\nAfter The Non Preemptive SJF\n");
    avgwt = findWaitingTime(bt, wt, n) / n;
    avgtat = findTurnAroundTime(bt, wt, tat, n) / n;

    printf("P|\tBT|\tWT|\tTAT\n");
    for(i = 0; i < n; i++)
    {
        printf("%d", p[i]);
        printf("\t%d", bt[i]);
        printf("\t%d", wt[i]);
        printf("\t%d\n", tat[i]);
    }

    printf("\nGantt Chart\n");
```

```
    for(i = 0; i < n; i++)
    {
        printf("| P%d ", i);
    }
    printf("|\n");

    for(i = 0; i < n; i++)
    {
        printf("%d ", wt[i]);
    }
    printf("%d\n", tat[i-1]);

    printf("Average Waiting Time= %d\n", avgwt);
    printf("Avegare Turn Around Time= %d", avgtat);
}
```

SLIP 10

Q.1 Write the program to simulate Round Robin (RR) scheduling. The arrival time and first CPU burst for different n number of processes should be input to the algorithm. Also give the time quantum as input. The next CPU burst should be generated randomly. The output should give Gantt chart, turnaround time and waiting time for each process. Also find the average waiting time and turnaround time.

```
#include <stdio.h>

int findWaitingTime(int bt[], int wt[], int n, int tq)
{
    int rem[10];
    int time = 0;
    int i, done;

    for(i = 0; i < n; i++)
    {
        rem[i] = bt[i];
        wt[i] = 0;
    }

    do
    {
        done = 1;
        for(i = 0; i < n; i++)
```

```
        {
            if(rem[i] > 0)
            {
                done = 0;
                if(rem[i] > tq)
                {
                    time = time + tq;
                    rem[i] = rem[i] - tq;
                }
                else
                {
                    time = time + rem[i];
                    rem[i] = 0;
                    wt[i] = time - bt[i];
                }
            }
        }
    } while(done == 0);

    return 0;
}

int findTurnAroundTime(int bt[], int wt[], int tat[], int n)
{
    int tatavg = 0;
    int i;

    for(i = 0; i < n; i++)
    {
        tat[i] = bt[i] + wt[i];
        tatavg = tatavg + tat[i];
    }

    return tatavg;
}

void ganttChart(int bt[], int n, int tq)
{
    int rem[10];
    int process[100];
    int time[100];
    int count = 0;
    int currentTime = 0;
    int i, done;
```

```
for(i = 0; i < n; i++)
{
   rem[i] = bt[i];
}

do
{
   done = 1;
   for(i = 0; i < n; i++)
   {
      if(rem[i] > 0)
      {
         done = 0;
         process[count] = i;
         if(rem[i] > tq)
         {
            currentTime = currentTime + tq;
            rem[i] = rem[i] - tq;
         }
         else
         {
            currentTime = currentTime + rem[i];
            rem[i] = 0;
         }
         time[count] = currentTime;
         count++;
      }
   }
} while(done == 0);

printf("\nGantt Chart\n");
for(i = 0; i < count; i++)
{
   printf("| P%d ", process[i] + 1);
}
printf("|\n");
printf("0");

for(i = 0; i < count; i++)
{
   printf("\t%d", time[i]);
}
printf("\n");
```

```
}

int main()
{
    int bt[10], wt[10], tat[10];
    int p[10];
    int i, n, tq;
    int avgwt, avgtat;

    printf("Enter the Number of Process\n");
    scanf("%d", &n);
    printf("Enter the Time Quantum\n");
    scanf("%d", &tq);
    printf("Enter the Burst Time\n");

    for(i = 0; i < n; i++)
    {
        p[i] = i + 1;
        printf("\nProcess %d: ", p[i]);
        scanf("%d", &bt[i]);
        printf("\n");
    }

    findWaitingTime(bt, wt, n, tq);
    avgtat = findTurnAroundTime(bt, wt, tat, n);

    printf("\nProcess\tBT\tWT\tTAT\n");
    for(i = 0; i < n; i++)
    {
        printf("P%d\t%d\t%d\t%d\n",p[i], bt[i], wt[i], tat[i]);
    }

    avgwt = 0;
    for(i = 0; i < n; i++)
    {
        avgwt = avgwt + wt[i];
    }

    printf("\nAverage Waiting Time = %.2f",(float)avgwt / n);
    printf("\nAverage Turn Around Time = %.2f",(float)avgtat / n);
    printf("\n");

    ganttChart(bt, n, tq);
```

```
    return 0;
}
```

Q.2 Write a simulation program for disk scheduling using Look algorithm. Accept total number of disk blocks,
disk request string, and current head position from the user. Display the list of request in the order in which it is
served. Also display the total number of head moments. 55, 58, 39, 18, 90, 160, 150, 38.
Start Head Position: 100. Direction: Left

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int n, i, j, head, direction;
    int request[100], temp;
    int totalMovement = 0;
    int current;

    printf("Enter total number of disk blocks: ");
    scanf("%d", &n);

    printf("Enter number of disk requests: ");
    scanf("%d", &j);

    printf("Enter disk request string:\n");
    for (i = 0; i < j; i++)
    {
        scanf("%d", &request[i]);
    }

    printf("Enter current head position: ");
    scanf("%d", &head);

    printf("Enter direction (0 for Left, 1 for Right): ");
    scanf("%d", &direction);

    for (i = 0; i < j - 1; i++)
    {
        for (int k = 0; k < j - i - 1; k++)
        {
            if (request[k] > request[k + 1])
            {
```

```
            temp = request[k];
            request[k] = request[k + 1];
            request[k + 1] = temp;
        }
    }
}

current = head;
printf("\nRequest service order:\n%d", head);

if (direction == 0)
{

    for (i = j - 1; i >= 0; i--)
    {
        if (request[i] < head)
        {
            printf(" -> %d", request[i]);
            totalMovement += abs(current - request[i]);
            current = request[i];
        }
    }

    for (i = 0; i < j; i++)
    {
        if (request[i] >= head)
        {
            printf(" -> %d", request[i]);
            totalMovement += abs(current - request[i]);
            current = request[i];
        }
    }
}
else
{
    /* Service requests to the right */
    for (i = 0; i < j; i++)
    {
        if (request[i] > head)
        {
            printf(" -> %d", request[i]);
            totalMovement += abs(current - request[i]);
            current = request[i];
        }
```

```
        }

        for (i = j - 1; i >= 0; i--)
        {
            if (request[i] <= head)
            {
                printf(" -> %d", request[i]);
                totalMovement += abs(current - request[i]);
                current = request[i];
            }
        }
    }

    printf("\nTotal head movement = %d blocks\n", totalMovement);

    return 0;
}
```

SLIP 11

Q.1) Write a C Menu driven Program to implement following functionality
a) Accept Available
b) Display Allocation, Max
c) Display the contents of need matrix
d) Display Available

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 10

int max[MAX][MAX], need[MAX][MAX], allocation[MAX][MAX];
int available[MAX], work[MAX], finish[MAX], request[MAX], safeSeq[MAX];
int np, nr;

void accept_matrix(int A[MAX][MAX])
{
    int i, j;
    for(i = 0; i < np; i++)
        for(j = 0; j < nr; j++)
            scanf("%d", &A[i][j]);
}
```

```
void accept_vector(int A[MAX])
{
    int i;
    for(i = 0; i < nr; i++)
        scanf("%d", &A[i]);
}

void calculate_need()
{
    int i, j;
    for(i = 0; i < np; i++)
        for(j = 0; j < nr; j++)
            need[i][j] = max[i][j] - allocation[i][j];
    printf("\nNeed matrix calculated successfully.\n");
}

void display()
{
    int i, j;
    printf("\nAllocation\tMax\t\tNeed\n");
    for(i = 0; i < np; i++)
    {
        for(j = 0; j < nr; j++)
            printf("%d ", allocation[i][j]);
        printf("\t\t");

        for(j = 0; j < nr; j++)
            printf("%d ", max[i][j]);

        printf("\t\t");
        for(j = 0; j < nr; j++)
            printf("%d ", need[i][j]);
        printf("\n");
    }
    printf("\nAvailable: ");
    for(i = 0; i < nr; i++)
        printf("%d ", available[i]);
    printf("\n");
}

int main()
{
    int choice;
```

```
    printf("Enter number of processes: ");
    scanf("%d", &np);
    printf("Enter number of resources: ");
    scanf("%d", &nr);

    do
    {
        printf("\n----- Menu-----\n");
        printf("1. Accept Allocation\n");
        printf("2. Accept Max\n");
        printf("3. Calculate Need\n");
        printf("4. Accept Available\n");
        printf("5. Display Matrices\n");
        printf("6. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch(choice)
        {
            case 1:
                printf("Enter Allocation Matrix:\n");
                accept_matrix(allocation);
                break;
            case 2:
                printf("Enter Max Matrix:\n");
                accept_matrix(max);
                break;
            case 3:
                calculate_need();
                break;
            case 4:
                printf("Enter Available Vector:\n");
                accept_vector(available);
                break;
            case 5:
                display();
                break;
        }
    } while(choice != 6);

    return 0;
}
```

Q.2 Write the simulation program for demand paging and show the page scheduling and total number of page
faults according the optimal page replacement algorithm. Assume the memory of n frames. Reference String: 8, 5, 7, 8, 5, 7, 2, 3, 7, 3, 5, 9, 4, 6, 2

```
#include<stdio.h>

int main()
{
   int no_of_frames, no_of_pages, frames[10], pages[30], temp[10];
   int fund, freeSlot, neverUsed, i, j, k, pos, max, faults = 0;

   print("Enter number of frames: ");
   scanf("%d", &no_of_frames);

   printf("Enter number of pages: ");
   scanf("%d", &no_of_pages);

   printf("Enter page reference string: ");
   for(i = 0; i < no_of_pages; ++i)
   {
      scanf("%d", &pages[i]);
   }

   for(i = 0; i < no_of_frames; ++i)
   {
      frames[i] = -1;
   }

   printf("\nPage Replacement Process:\n");
   for(i = 0; i < no_of_pages; ++i)
   {
      found = freeSlot = 0;

      for(j = 0; j < no_of_frames; ++j)
      {
         if(frames[j] == pages[i])
         {
            found = 1;
            break;
         }
      }

      if(!found)
```

```
{
   for(j = 0; j < no_of_frames; ++j)
   {
      if(frames[j] == -1)
      {
         frames[j] = pages[i];
         faults++;
         freeSlot = 1;
         break;
      }
   }
}

if(!found && !freeSlot)
{
   for(j = 0; j < no_of_frames; ++j)
   {
      temp[j] = -1;
      for(k = i+1; k < no_of_pages; ++k)
      {
         if(frames[j] == pages[k])
         {
            temp[j] = k;
            break;
         }
      }
   }

   neverUsed = 0;
   for(j = 0; j < no_of_frames; ++j)
   {
      if(temp[j] == -1)
      {
         pos = j;
         neverUsed = 1;
         break;
      }
   }

   if(!neverUsed)
   {
      max = temp[0];
      pos = 0;
      for(j = 1; j < no_of_frames; ++j)
```

```
            {
                if(temp[j] > max)
                {
                    max = temp[j];
                    pos = j;
                }
            }
        }

        frames[pos] = pages[i];
        faults++;
    }

    printf("Page %2d -> ", pages[i]);
    for(j = 0; j < no_of_frames; ++j)
    {
        if(frames[j] != -1)
            printf("%d ", frames[j]);
        else
            printf("- ");
    }
    printf("\n");
  }

  printf("\nTotal Page Faults = %d\n", faults);

  return 0;
}
```

SLIP 12

Q.1 Write the program to simulate FCFS CPU scheduling. The arrival time and first CPU-burst for different n number of processes should be input to the algorithm. The next CPU-burst should be generated randomly. The output should give a Gantt chart, turnaround time and waiting time for each process. Also find the average waiting time and turnaround time.

```
#include <stdio.h>

int findWaitingTime(int bt[], int wt[], int n)
{
  int wtavg = 0, i;
  wt[0] = 0;
```

```
    for(i = 1; i < n; i++)
    {
        wt[i] = bt[i-1] + wt[i-1] + 2;
        wtavg = wtavg + wt[i];
    }
    return(wtavg);
}

int findTurnAroundTime(int bt[], int wt[], int tat[], int n)
{
    int tatavg = 0, i;
    for (i = 0; i < n; i++)
    {
        tat[i] = bt[i] + wt[i];
        tatavg = tatavg + tat[i];
    }
    return(tatavg);
}

void main()
{
    int at[10], bt[10], wt[10], tat[10], i, n;
    int avgwt, avgtat;

    printf("\nEnter the number of process\n");
    scanf("%d", &n);

    for(i = 0; i < n; i++)
    {
        bt[i] = rand() % 10;
    }

    for(i = 0; i < n; i++)
    {
        printf("%d\t", bt[i]);
    }

    avgwt = findWaitingTime(bt, wt, n) / n;
    avgtat = findTurnAroundTime(bt, wt, tat, n) / n;

    printf("\n\nP|\tBT\tWT\tTAT|\n");
    for(i = 0; i < n; i++)
    {
        printf("%d", i);
```

```
        printf("\t%d", bt[i]);
        printf("\t%d", wt[i]);
        printf("\t%d\n", tat[i]);
    }

    printf("\nGantt Chart\n");
    for(i = 0; i < n; i++)
    {
        printf("| P%d\t", i);
    }
    printf("|\n");

    for(i = 0; i < n; i++)
    {
        printf("%d\t ", wt[i]);
    }
    printf("%d\n", tat[i-1]);

    printf("Average Waiting Time=%d", avgwt);
    printf("\n");
    printf("Average Turn Around Time=%d", avgtat);
}
```

Q.2 Implement the C program to accept n integers to be sorted. Main function creates child process using a fork system call. Parent process sorts the integers using bubble sort and waits for child process using wait system call child process sorts the integers using insertion sort.

```
#include<stdio.h>
#include<unistd.h>
#include<sys/types.h>
#include<sys/wait.h>

void bubble_sort(int [], int);
void insertion_sort(int [], int);

int main()
{
    int i, j, n, *status = NULL, arr[30];
    pid_t pid;

    printf("\nEnter the number of elements:");
    scanf("%d", &n);
```

```
    for(i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    pid = fork();

    if(pid == 0)
    {
        printf("\n\t This is child process. ");
        printf("\n\t My process id is : %d", getpid());
        printf("\n\t My Parent process id is : %d", getppid());

        insertion_sort(arr, n);

        printf("\nInsertion ort");
        for(i = 0; i < n; i++)
            printf(" %d", arr[i]);

        printf("\n\n");
    }
    else
    {
        printf("\n\n\t Parent process resumed after the execution of child process with PID %d",
pid);
        printf("\n\t My process id is : %d", getpid());
        printf("\n\t My Parent process id is : %d", getppid());

        bubble_sort(arr, n);

        printf("\nBubble Sort:");
        for(i = 0; i < n; i++)
            printf(" %d", arr[i]);

        printf("\n\n");
        pid = wait(status);
    }
}

void bubble_sort(int *a, int n)
{
    int i, j, temp;
    for(i = 0; i < n; i++)
    {
```

```
        for(j = 0; j < n - i - 1; j++)
        {
            if(a[j] > a[j + 1])
            {
                temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }
        }
    }
}

void insertion_sort(int *a, int n)
{
    int i, j, temp;
    for(i = 1; i < n; i++)
    {
        temp = a[i];
        for(j = i - 1; j >= 0 && temp < a[j]; j--)
            a[j + 1] = a[j];

        a[j + 1] = temp;
    }
}
```

SLIP 13

Q.1 Write the program to simulate Non-preemptive Shortest Job First (SJF) scheduling. The arrival time and first CPU burst for different n number of processes should be input to the algorithm. The next CPU-burst should be generated randomly. The output should give Gantt chart, turnaround time and waiting time for each process.Also find the average waiting time and turnaround time.

```
#include<stdio.h>

int findWaitingTime(int bt[], int wt[], int n)
{
    int wtavg = 0, i;
    wt[0] = 0;
    for(i = 1; i < n; i++)
    {
        wt[i] = bt[i-1] + wt[i-1] + 2;
        wtavg = wtavg + wt[i];
    }
```

```
    return(wtavg);
}

int findTurnAroundTime(int bt[], int wt[], int tat[], int n)
{
    int tatavg = 0, i;
    for(i = 0; i < n; i++)
    {
        tat[i] = bt[i] + wt[i];
        tatavg = tatavg + tat[i];
    }
    return(tatavg);
}

void sort(int n, int bt[], int p[])
{
    int i, j, pos, temp;
    for(i = 0; i < n; i++)
    {
        pos = i;
        for(j = i + 1; j < n; j++)
        {
            if(bt[j] < bt[pos])
            {
                pos = j;
            }
        }

        temp = at[i];
        at[i] = at[pos];
        at[pos] = temp;

        temp = bt[i];
        bt[i] = bt[pos];
        bt[pos] = temp;

        temp = p[i];
        p[i] = p[pos];
        p[pos] = temp;
    }
}

void main()
{
```

```
int at[10], bt[10], wt[10], tat[10], p[10], i, n;
int avgwt, avgtat;

printf("\nEnter The Number of Process\n");
scanf("%d", &n);

printf("\nEnter The Arrival and Burst Time\n");
for(i = 0; i < n; i++)
{
    p[i] = i;
    printf("\nProcess %d:\t", p[i]);
    scanf("%d%d", &at[i], &bt[i]);
    printf("\n");
}

printf("\nBefore Sorting\n");
printf("P|\tBT|\n");
for(i = 0; i < n; i++)
{
    printf("%d", p[i]);
    printf("\t%d\n", bt[i]);
}

sort(n, bt, p);

printf("\nAfter The Non Preemptive SJF\n");
avgwt = findWaitingTime(bt, wt, n) / n;
avgtat = findTurnAroundTime(bt, wt, tat, n) / n;

printf("P|\tBT|\tWT|\tTAT\n");
for(i = 0; i < n; i++)
{
    printf("%d", p[i]);
    printf("\t%d", bt[i]);
    printf("\t%d", wt[i]);
    printf("\t%d\n", tat[i]);
}

printf("\nGantt Chart\n");
for(i = 0; i < n; i++)
{
    printf("| P%d ", i);
}
printf("|\n");
```

```
    for(i = 0; i < n; i++)
    {
        printf("%d ", wt[i]);
    }
    printf("%d\n", tat[i-1]);

    printf("Average Waiting Time= %d\n", avgwt);
    printf("Avegare Turn Around Time= %d", avgtat);
}
```

Q.2 Write a simulation program for disk scheduling using LOOK algorithm. Accept total number of disk blocks, disk request string, and current head position from the user. isplay the list of requests in the order in which it is served. Also display the total number of head moments 15, 30, 55, 90, 125, 140, 170 Starting Head Position: 100. Direction: Left

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int n, i, j, head, direction;
    int request[100], temp;
    int totalMovement = 0;
    int current;

    printf("Enter total number of disk blocks: ");
    scanf("%d", &n);

    printf("Enter number of disk requests: ");
    scanf("%d", &j);

    printf("Enter disk request string:\n");
    for (i = 0; i < j; i++)
    {
        scanf("%d", &request[i]);
    }

    printf("Enter current head position: ");
    scanf("%d", &head);

    printf("Enter direction (0 for Left, 1 for Right): ");
    scanf("%d", &direction);
```

```
for (i = 0; i < j - 1; i++)
{
    for (int k = 0; k < j - i - 1; k++)
    {
        if (request[k] > request[k + 1])
        {
            temp = request[k];
            request[k] = request[k + 1];
            request[k + 1] = temp;
        }
    }
}

current = head;
printf("\nRequest service order:\n%d", head);

if (direction == 0)
{
    for (i = j - 1; i >= 0; i--)
    {
        if (request[i] < head)
        {
            printf(" -> %d", request[i]);
            totalMovement += abs(current - request[i]);
            current = request[i];
        }
    }
    for (i = 0; i < j; i++)
    {
        if (request[i] >= head)
        {
            printf(" -> %d", request[i]);
            totalMovement += abs(current - request[i]);
            current = request[i];
        }
    }
}
else
{
    for (i = 0; i < j; i++)
    {
        if (request[i] > head)
        {
            printf(" -> %d", request[i]);
```

```
            totalMovement += abs(current - request[i]);
            current = request[i];
        }
    }

    for (i = j - 1; i >= 0; i--)
    {
        if (request[i] <= head)
        {
            printf(" -> %d", request[i]);
            totalMovement += abs(current - request[i]);
            current = request[i];
        }
    }
}

printf("\nTotal head movement = %d blocks\n", totalMovement);

return 0;
}
```

SLIP 16

Q.1 Write the simulation program to implement demand paging and show the page scheduling and total number of page faults according to the LRU (using counter method) page replacement algorithm. Assume the memory of n frames.
Reference String : 3,5,7,2,5,1,2,3,1,3,5,3,1,6,2

```
#include <stdio.h>
#define MAX 20

int frames[MAX], ref[MAX], mem[MAX][MAX], faults, sp, m, n, time[MAX];

void accept()
{
    int i;
    printf("Enter no.of frames: ");
    scanf("%d", &n);
    printf("Enter no.of references: ");
    scanf("%d", &m);
    printf("Enter reference string:\n");
    for(i = 0; i < m; i++)
    {
        printf("[%d]=", i);
```

```
        scanf("%d", &ref[i]);
    }
}

void disp()
{
    int i, j;
    printf("\nReference string:\n");
    for(i = 0; i < m; i++)
        printf("%3d", ref[i]);

    printf("\n\n");
    for(i = 0; i < n; i++)
    {
        for(j = 0; j < m; j++)
        {
            if(mem[i][j] != -1)
                printf("%3d", mem[i][j]);
            else
                printf(" ");
        }
        printf("\n");
    }
    printf("Total Page Faults: %d\n", faults);
}

int search(int pno)
{
    int i;
    for(i = 0; i < n; i++)
    {
        if(frames[i] == pno)
            return i;
    }
    return -1;
}

int get_lru()
{
    int i, min_i = 0, min = 9999;
    for(i = 0; i < n; i++)
    {
        if(time[i] < min)
        {
```

```
            min = time[i];
            min_i = i;
        }
    }
    return min_i;
}

void lru()
{
    int i, j, k;
    for(i = 0; i < m && sp < n; i++)
    {
        k = search(ref[i]);
        if(k == -1)
        {
            frames[sp] = ref[i];
            time[sp] = i;
            faults++;
            sp++;
        }
        else
        {
            time[k] = i;
        }

        for(j = 0; j < n; j++)
            mem[j][i] = frames[j];
    }

    for(; i < m; i++)
    {
        k = search(ref[i]);
        if(k == -1)
        {
            sp = get_lru();
            frames[sp] = ref[i];
            time[sp] = i;
            faults++;
        }
        else
            time[k] = i;

        for(j = 0; j < n; j++)
            mem[j][i] = frames[j];
```

```
    }
}

int main()
{
    int i, j;

    for(i = 0; i < MAX; i++)
    {
        frames[i] = -1;
        time[i] = 0;
        for(j = 0; j < MAX; j++)
            mem[i][j] = -1;
    }

    faults = sp = 0;

    accept();
    lru();
    disp();

    return 0;
}
```

Q.2 Write a C program that behaves like a shell which displays the command prompt '$'. It accepts the command, tokenize the command line and execute it by creating the child process. Also implement the additional command 'count' as
a $ count c filename: It will display the number of characters in given file
b $ count w filename: It will display the number of words in given file
c $ count l filename: It will display the number of lines in given file

```
#include <sys/types.h>
#include <sys/stat.h>
#include <sys/wait.h>
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <dirent.h>
#include <unistd.h>

void make_toks(char *s, char *tok[])
{
    int i = 0;
```

```
    char *p;

    p = strtok(s, " ");
    while(p != NULL)
    {
        tok[i++] = p;
        p = strtok(NULL, " ");
    }
    tok[i] = NULL;
}

void count(char *fn, char op)
{
    int fh, cc = 0, wc = 0, lc = 0;
    char c;

    fh = open(fn, O_RDONLY);
    if(fh == -1)
    {
        printf("File %s not found.\n", fn);
        return;
    }

    while(read(fh, &c, 1) > 0)
    {
        if(c == ' ')
            wc++;
        else if(c == '\n')
        {
            wc++;
            lc++;
        }
        cc++;
    }
    close(fh);

    switch(op)
    {
        case 'c':
            printf("No.of characters:%d\n", cc);
            break;
        case 'w':
            printf("No.of words:%d\n", wc);
            break;
```

```
        case 'l':
            printf("No.of lines:%d\n", lc);
            break;
    }
}

int main()
{
    char buff[80], *args[10];
    int pid;

    while(1)
    {
        printf("myshell$");
        fflush(stdin);
        fgets(buff, 80, stdin);
        buff[strlen(buff)-1] = '\0';

        make_toks(buff, args);

        if(strcmp(args[0], "quit") == 0)
            break;
        else if(strcmp(args[0], "count") == 0)
            count(args[2], args[1][0]);
        else
        {
            pid = fork();
            if(pid > 0)
                wait(NULL);
            else
            {
                if(execvp(args[0], args) == -1)
                    printf("Bad command.\n");
            }
        }
    }
    return 0;
}
```

SLIP 20

Q.1 Write a simulation program for disk scheduling using LOOK algorithm. Accept total number of disk blocks, disk request string, and current head position from the user. Display the list of requests in the order in which it

is served. Also display the total number of head moments.
86, 147, 91, 177, 45, 12, 130
Starting Head Position: 60 Direction: Right

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
   int RQ[100], i, j, n, TotalHeadMoment = 0;
   int initial, size, move, temp, index;

   printf("Enter total disk size: ");
   scanf("%d", &size);

   printf("Enter the number of Requests: ");
   scanf("%d", &n);

   printf("Enter the Requests sequence:\n");
   for(i = 0; i < n; i++)
      scanf("%d", &RQ[i]);

   printf("Enter initial head position: ");
   scanf("%d", &initial);

   printf("Enter the head movement direction (1 for Right/High, 0 for Left/Low): ");
   scanf("%d", &move);

   for(i = 0; i < n; i++)
   {
      for(j = 0; j < n - i - 1; j++)
      {
         if(RQ[j] > RQ[j+1])
         {
            temp = RQ[j];
            RQ[j] = RQ[j+1];
            RQ[j+1] = temp;
         }
      }
   }

   for(i = 0; i < n; i++)
   {
      if(initial < RQ[i])
```

```
    {
        index = i;
        break;
    }
}

printf("\nRequest service order:\n%d", initial);

if(move == 1)
{

    for(i = index; i < n; i++)
    {
        printf(" -> %d", RQ[i]);
        TotalHeadMoment = TotalHeadMoment + abs(RQ[i] - initial);
        initial = RQ[i];
    }

    for(i = index - 1; i >= 0; i--)
    {
        printf(" -> %d", RQ[i]);
        TotalHeadMoment = TotalHeadMoment + abs(RQ[i] - initial);
        initial = RQ[i];
    }
}
// If movement is towards Left (Low value)
else
{

    for(i = index - 1; i >= 0; i--)
    {
        printf(" -> %d", RQ[i]);
        TotalHeadMoment = TotalHeadMoment + abs(RQ[i] - initial);
        initial = RQ[i];
    }

    for(i = index; i < n; i++)
    {
        printf(" -> %d", RQ[i]);
        TotalHeadMoment = TotalHeadMoment + abs(RQ[i] - initial);
        initial = RQ[i];
    }
}
```

```
    printf("\n\nTotal head movement is %d\n", TotalHeadMoment);

    return 0;
}
```

Q.2 Write the simulation program to implement demand paging and show the page scheduling and total number of page faults according to the LRU (using counter method) page replacement algorithm. Assume the memory of n frames. Reference String : 3,5,7,2,5,1,2,3,1,3,5,3,1,6,2

```
#include <stdio.h>

int main()
{
    int no_of_frames, frames[10], time[10];
    int ref[] = {3, 5, 7, 2, 5, 1, 2, 3, 1, 3, 5, 3, 1, 6, 2};
    int no_of_pages = sizeof(ref) / sizeof(ref[0]);
    int faults = 0, counter = 0;
    int i, j, found, pos, min;

    printf("Enter number of frames: ");
    scanf("%d", &no_of_frames);

    for(i = 0; i < no_of_frames; ++i)
    {
        frames[i] = -1;
        time[i] = 0;
    }

    printf("\nReference String: ");
    for(i = 0; i < no_of_pages; i++)
    {
        printf("%d ", ref[i]);
    }

    printf("\n\nPage Replacement Process (LRU):\n");
    for(i = 0; i < no_of_pages; ++i)
    {
        found = 0;
        counter++;
        for(j = 0; j < no_of_frames; ++j)
        {
            if(frames[j] == ref[i])
```

```
    {
        found = 1;
        time[j] = counter; // Update time counter on page hit
        break;
    }
}

if(!found)
{
    pos = -1;

    // Check for empty slot first
    for(j = 0; j < no_of_frames; ++j)
    {
        if(frames[j] == -1)
        {
            pos = j;
            break;
        }
    }

    if(pos == -1)
    {
        min = time[0];
        pos = 0;
        for(j = 1; j < no_of_frames; ++j)
        {
            if(time[j] < min)
            {
                min = time[j];
                pos = j;
            }
        }
    }

    frames[pos] = ref[i];
    time[pos] = counter;
    faults++;
}

printf("Page %d -> ", ref[i]);
for(j = 0; j < no_of_frames; ++j)
{
    if(frames[j] != -1)
```

```
            printf("%d ", frames[j]);
        else
            printf("- ");
    }

    if(!found)
        printf("\t(Fault)");

    printf("\n");
  }

  printf("\nTotal Page Faults = %d\n", faults);

  return 0;
}
```

SLIP 23

Q.1 Implement the C program to accept n integers to be sorted. Main function creates child process using fork system call. Parent process sorts the integers using bubble sort and waits for child process using wait system call. Child process sorts the integers using insertion sort

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

void bubble_sort(int [], int);
void insertion_sort(int [], int);

int main()
{
  int i, n, arr[30];
  int *status = NULL;
  pid_t pid;

  printf("\nEnter the number of elements: ");
  scanf("%d", &n);

  printf("Enter the elements: ");
  for(i = 0; i < n; i++)
  {
    scanf("%d", &arr[i]);
```

```
    }

    pid = fork();

    if(pid == 0)
    {
        printf("\n\t This is child process.");
        printf("\n\t My process id is : %d", getpid());
        printf("\n\t My Parent process id is : %d", getppid());

        insertion_sort(arr, n);

        printf("\n\nInsertion Sort (Child):");
        for(i = 0; i < n; i++)
            printf(" %d", arr[i]);

        printf("\n");
    }
    else if(pid > 0)
    {
        printf("\n\t Parent process resumed after the execution of child process with PID %d", pid);
        printf("\n\t My process id is : %d", getpid());
        printf("\n\t My Parent process id is : %d", getppid());

        bubble_sort(arr, n);

        printf("\n\nBubble Sort (Parent):");
        for(i = 0; i < n; i++)
            printf(" %d", arr[i]);

        printf("\n\n");
        pid = wait(status);
    }
    else
    {
        printf("Failed to create child process");
    }

    return 0;
}

void bubble_sort(int *a, int n)
{
    int i, j, temp;
```

```
    for(i = 0; i < n; i++)
    {
        for(j = 0; j < n - i - 1; j++)
        {
            if(a[j] > a[j + 1])
            {
                temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }
        }
    }
}

void insertion_sort(int *a, int n)
{
    int i, j, temp;
    for(i = 1; i < n; i++)
    {
        temp = a[i];
        for(j = i - 1; j >= 0 && temp < a[j]; j--)
            a[j + 1] = a[j];

        a[j + 1] = temp;
    }
}
```

Q.2 Write the program to simulate Non-preemptive Shortest Job First (SJF) scheduling. The arrival time and first CPU-burst for different n number of processes should be input to the algorithm. The next CPU-burst should be generated randomly. The output should give Gantt chart, turnaround time and waiting time for each process. Also find the average waiting time and turnaround time.

```
#include <stdio.h>
#include <stdlib.h>

int findWaitingTime(int bt[], int wt[], int n)
{
    int wtavg = 0, i;
    wt[0] = 0;
    for(i = 1; i < n; i++)
    {
```

```
        // Waiting time for next process is BT + WT of previous process
        wt[i] = bt[i-1] + wt[i-1];
        wtavg = wtavg + wt[i];
    }
    return(wtavg);
}

int findTurnAroundTime(int bt[], int wt[], int tat[], int n)
{
    int tatavg = 0, i;
    for(i = 0; i < n; i++)
    {
        tat[i] = bt[i] + wt[i];
        tatavg = tatavg + tat[i];
    }
    return(tatavg);
}

void sort(int n, int bt[], int p[], int at[])
{
    int i, j, pos, temp;
    for(i = 0; i < n; i++)
    {
        pos = i;
        for(j = i + 1; j < n; j++)
        {
            if(bt[j] < bt[pos])
            {
                pos = j;
            }
        }

        // Swap Arrival Time
        temp = at[i];
        at[i] = at[pos];
        at[pos] = temp;

        // Swap Burst Time
        temp = bt[i];
        bt[i] = bt[pos];
        bt[pos] = temp;

        // Swap Process ID
        temp = p[i];
```

```
        p[i] = p[pos];
        p[pos] = temp;
    }
}

int main()
{
    int at[10], bt[10], next_bt[10], wt[10], tat[10], p[10], i, n;
    int avgwt, avgtat;

    printf("\nEnter The Number of Process: ");
    scanf("%d", &n);

    printf("\nEnter The Arrival and First Burst Time\n");
    for(i = 0; i < n; i++)
    {
        p[i] = i;
        printf("Process %d: ", p[i]);
        scanf("%d %d", &at[i], &bt[i]);

        // Generate the next CPU-burst randomly as per the question
        next_bt[i] = rand() % 10 + 1;
    }

    printf("\nBefore Sorting\n");
    printf("P|\tAT|\tFirst BT|\tNext BT(Random)|\n");
    for(i = 0; i < n; i++)
    {
        printf("%d\t%d\t%d\t\t%d\n", p[i], at[i], bt[i], next_bt[i]);
    }

    // Sort processes according to burst time (Non-preemptive SJF logic)
    sort(n, bt, p, at);

    avgwt = findWaitingTime(bt, wt, n);
    avgtat = findTurnAroundTime(bt, wt, tat, n);

    printf("\nAfter The Non Preemptive SJF Scheduling\n");
    printf("P|\tBT|\tWT|\tTAT\n");
    for(i = 0; i < n; i++)
    {
        printf("%d\t%d\t%d\t%d\n", p[i], bt[i], wt[i], tat[i]);
    }
```

```
    printf("\nGantt Chart\n");
    for(i = 0; i < n; i++)
    {
        printf("| P%d\t", p[i]);
    }
    printf("|\n");

    for(i = 0; i < n; i++)
    {
        printf("%d\t", wt[i]);
    }
    // Print total completion time at the end
    printf("%d\n", tat[n-1] + at[n-1]);

    printf("\nAverage Waiting Time = %.2f", (float)avgwt / n);
    printf("\nAverage Turn Around Time = %.2f\n", (float)avgtat / n);

    return 0;
}
```

SLIP 25

Q.1 Write a C program to illustrate the concept of orphan process. The parent process creates a child and terminates before the child has finished its task. So the child process becomes an orphan process. (Use fork(), sleep(), getpid(), getppid()

```
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>

int main()
{
    int pid = fork();

    if(id > 0)
    {
        printf("\nIAM PARENT PROCESS,MY PROCESS ID IS: %d", getpid());
        printf("\n THE PARENT'S PARENT PROCESS ID IS : %d\n", getppid());
    }
    else if(pid == 0)
    {
```

```
        printf("\n In Child process\n");
        printf("\n IAM CHILD PROCESS,MY PROCESS ID IS: %d", getpid());
        printf("\n THE CHILDS'S PARENT PROCESSS ID IS : %d\n", getppid());

        sleep(10);

        printf("\n —--After 10 Seconds—----");
        printf("\n IAM CHILD PROCESS,MY PROCESS ID IS: %d", getpid());
        printf("\n THE CHILDS'S PARENT PROCESS ID IS :%d\n", getppid());
    }
    else
    {
        printf("Failed to create child process");
    }

    return 0;
}
```

Q.2 Write a C program that behaves like a shell which displays the command prompt '$'. It accepts the command, tokenize the command line and execute it by creating the child process. Also implement the additional command 'search' as
a. $ search f filename pattern : It will search the first occurrence of pattern in the given file
b. $ search a filename pattern : It will search all the occurrence of pattern in the given file
c. $ search c filename pattern : It will count the number of occurrence of pattern in the given file

```
#include <sys/types.h>
#include <sys/stat.h>
#include <sys/wait.h>
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <dirent.h>
#include <unistd.h>

void make_toks(char *s, char *tok[])
{
    int i = 0;
    char *p;

    p = strtok(s, " ");
    while(p != NULL)
```

```
    {
        tok[i++] = p;
        p = strtok(NULL, " ");
    }
    tok[i] = NULL;
}

void search(char *fn, char op, char *pattern)
{
    int fh, count = 0, i = 0, j = 0;
    char buff[255], c, *p;

    fh = open(fn, O_RDONLY);
    if(fh == -1)
    {
        printf("File %s Not Found\n", fn);
        return;
    }

    switch(op)
    {
        case 'f':
            while(read(fh, &c, 1))
            {
                buff[j++] = c;
                if(c == '\n')
                {
                    buff[j] = '\0';
                    j = 0;
                    i++;
                    if(strstr(buff, pattern))
                    {
                        printf("%d: %s", i, buff);
                        break;
                    }
                }
            }
            break;

        case 'c':
            while(read(fh, &c, 1))
            {
                buff[j++] = c;
                if(c == '\n')
```

```
            {
                buff[j] = '\0';
                j = 0;
                p = buff;
                while(p = strstr(p, pattern))
                {
                    count++;
                    p++;
                }
            }
        }
        printf("Total No.of Occurrences = %d\n", count);
        break;

    case 'a':
        while(read(fh, &c, 1))
        {
            buff[j++] = c;
            if(c == '\n')
            {
                buff[j] = '\0';
                j = 0;
                i++;
                if(strstr(buff, pattern))
                    printf("%d: %s", i, buff);
            }
        }
    }//switch

    close(fh);
}//search

int main()
{
    char buff[80], *args[10];
    int pid;

    while(1)
    {
        printf("myshell$");
        fflush(stdin);
        fgets(buff, 80, stdin);
        buff[strlen(buff)-1] = '\0';
```

```
        make_toks(buff, args);

        if(strcmp(args[0], "quit") == 0)
            break;
        else if(strcmp(args[0], "search") == 0)
            search(args[2], args[1][0], args[3]);
        else
        {
            pid = fork();
            if(pid > 0)
                wait(NULL);
            else
            {
                if(execvp(args[0], args) == -1)
                    printf("Bad command.\n");
            }
        }
    }

    return 0;
}
```

SLIP 27

Q.1 Write a simulation program for disk scheduling using SSTF algorithm. Accept total number of disk blocks, disk request string, and current head position from the user. Display the list of requests in the order in which it is served. Also display the total number of head moments.
30, 10, 60, 95, 120, 150, 175. Start Head Position: 50

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int RQ[100], i, n, TotalHeadMoment = 0, initial, count = 0;
    int min, diff, index, size;

    printf("Enter total number of disk blocks: ");
    scanf("%d", &size);

    printf("Enter the number of Requests: ");
    scanf("%d", &n);
```

```
printf("Enter the Requests sequence:\n");
for(i = 0; i < n; i++)
{
   scanf("%d", &RQ[i]);
}

printf("Enter initial head position: ");
scanf("%d", &initial);

printf("\nRequest service order:\n%d", initial);

// SSTF Logic
while(count != n)
{
   min = 10000; // Initialize with a very large value
   index = -1;

   // Find the closest request
   for(i = 0; i < n; i++)
   {
      if(RQ[i] != -1) // If request is not yet serviced
      {
         diff = abs(RQ[i] - initial);
         if(min > diff)
         {
            min = diff;
            index = i;
         }
      }
   }

   // Service the closest request found
   printf(" -> %d", RQ[index]);
   TotalHeadMoment = TotalHeadMoment + min;
   initial = RQ[index];

   // Mark as serviced
   RQ[index] = -1;
   count++;
}

printf("\n\nTotal head movement is %d blocks\n", TotalHeadMoment);

return 0;
```

}

Q.2 Write the simulation program to implement demand paging and show the page scheduling and total number of page faults according to the LRU (using counter method) page replacement algorithm. Assume the memory of n frames.
Reference String : 3,5,7,2,5,1,2,3,1,3,5,3,1,6,2

```
#include <stdio.h>

int main()
{
   int no_of_frames, frames[10], time[10];
   int ref[] = {3, 5, 7, 2, 5, 1, 2, 3, 1, 3, 5, 3, 1, 6, 2};
   int no_of_pages = sizeof(ref) / sizeof(ref[0]);
   int faults = 0, counter = 0;
   int i, j, found, pos, min;

   printf("Enter number of frames: ");
   scanf("%d", &no_of_frames);

   for(i = 0; i < no_of_frames; ++i)
   {
      frames[i] = -1;
      time[i] = 0;
   }

   printf("\nReference String: ");
   for(i = 0; i < no_of_pages; i++)
   {
      printf("%d ", ref[i]);
   }

   printf("\n\nPage Replacement Process (LRU):\n");
   for(i = 0; i < no_of_pages; ++i)
   {
      found = 0;
      counter++;

      for(j = 0; j < no_of_frames; ++j)
      {
         if(frames[j] == ref[i])
         {
            found = 1;
```

```
            time[j] = counter;
            break;
        }
    }

    if(!found)
    {
        pos = -1;

        for(j = 0; j < no_of_frames; ++j)
        {
            if(frames[j] == -1)
            {
                pos = j;
                break;
            }
        }

        if(pos == -1)
        {
            min = time[0];
            pos = 0;
            for(j = 1; j < no_of_frames; ++j)
            {
                if(time[j] < min)
                {
                    min = time[j];
                    pos = j;
                }
            }
        }

        frames[pos] = ref[i];
        time[pos] = counter;
        faults++;
    }

    printf("Page %d -> ", ref[i]);
    for(j = 0; j < no_of_frames; ++j)
    {
        if(frames[j] != -1)
            printf("%d ", frames[j]);
        else
            printf("- ");
```

```
        }

        if(!found)
            printf("\t(Fault)");

        printf("\n");
    }

    printf("\nTotal Page Faults = %d\n", faults);

    return 0;
}
```

SLIP 28

Q.1 Write a simulation program for disk scheduling using the SCAN algorithm. Accept total number of disk blocks, disk request string, and current head position from the user. Display the list of requests in the order in which it is served. Also display the total number of head moments.
10, 25, 75, 90, 130, 145, 180, 55. Starting Head position= 80. Direction: Left

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int RQ[100], i, j, n, TotalHeadMoment = 0;
    int initial, size, move, temp, index;

    printf("Enter total disk size\n");
    scanf("%d", &size);

    printf("Enter the number of Requests\n");
    scanf("%d", &n);

    printf("Enter the Requests sequence\n");
    for(i = 0; i < n; i++)
        scanf("%d", &RQ[i]);

    printf("Enter initial head position\n");
    scanf("%d", &initial);

    printf("Enter the head movement direction for high 1 and for low 0\n");
```

```
scanf("%d", &move);

/*logic for sort the request array */
for(i = 0; i < n; i++)
{
   for(j = 0; j < n - i - 1; j++)
   {
      if(RQ[j] > RQ[j+1])
      {
         temp = RQ[j];
         RQ[j] = RQ[j+1];
         RQ[j+1] = temp;
      }
   }
}

for(i = 0; i < n; i++)
{
   if(initial < RQ[i])
   {
      index = i;
      break;
   }
}

// if movement is towards high value
if(move == 1)
{
   for(i = index; i < n; i++)
   {
      TotalHeadMoment = TotalHeadMoment + abs(RQ[i] - initial);
      initial = RQ[i];
   }

   // last movement for max size
   TotalHeadMoment = TotalHeadMoment + abs(size - RQ[i-1] - 1);
   initial = size - 1;

   for(i = index - 1; i >= 0; i--)
   {
      TotalHeadMoment = TotalHeadMoment + abs(RQ[i] - initial);
      initial = RQ[i];
   }
}
```

```
    // if movement is towards low value
    else
    {
        for(i = index - 1; i >= 0; i--)
        {
            TotalHeadMoment = TotalHeadMoment + abs(RQ[i] - initial);
            initial = RQ[i];
        }

        //last movement for min size
        TotalHeadMoment = TotalHeadMoment + abs(RQ[i+1] - 0);
        initial = 0;

        for(i = index; i < n; i++)
        {
            TotalHeadMoment = TotalHeadMoment + abs(RQ[i] - initial);
            initial = RQ[i];
        }
    }

    printf("Total head movement is %d", TotalHeadMoment);

    return 0;
}
```

Q.2 Write the simulation program for demand paging and show the page scheduling and total number of page faults according the MFU page replacement algorithm. Assume the memory of n frames. Reference String: 8, 5, 7, 8, 5, 7, 2, 3, 7, 3, 5, 9, 4, 6, 2

```
#include <stdio.h>
#define MAX 20

int frames[MAX], ref[MAX], mem[MAX][MAX], faults, sp, m, n, count[MAX];

void accept()
{
    int i;
    printf("Enter no.of frames: ");
    scanf("%d", &n);
    printf("Enter no.of references: ");
    scanf("%d", &m);
    printf("Enter reference string:\n");
    for(i = 0; i < m; i++)
    {
```

```
        printf("[%d]=", i);
        scanf("%d", &ref[i]);
    }
}

void disp()
{
    int i, j;
    printf("\nReference string:\n");
    for(i = 0; i < m; i++)
        printf("%3d", ref[i]);

    printf("\n\n");

    for(i = 0; i < n; i++)
    {
        for(j = 0; j < m; j++)
        {
            if(mem[i][j] != -1)
                printf("%3d", mem[i][j]);
            else
                printf(" ");
        }
        printf("\n");
    }
    printf("Total Page Faults: %d\n", faults);
}

int search(int pno)
{
    int i;
    for(i = 0; i < n; i++)
    {
        if(frames[i] == pno)
            return i;
    }
    return -1;
}

int get_mfu(int sp)
{
    int i, max_i = 0, max = -9999;
    i = sp;
    do
```

```
    {
        if(count[i] > max)
        {
            max = count[i];
            max_i = i;
        }
        i = (i + 1) % n;
    } while(i != sp);

    return max_i;
}

void mfu()
{
    int i, j, k;

    for(i = 0; i < m && sp < n; i++)
    {
        k = search(ref[i]);
        if(k == -1)
        {
            frames[sp] = ref[i];
            count[sp] = 1;
            faults++;
            sp++;
        }
        else
        {
            count[k]++;
        }

        for(j = 0; j < n; j++)
            mem[j][i] = frames[j];
    }

    sp = 0;

    for(; i < m; i++)
    {
        k = search(ref[i]);
        if(k == -1)
        {
            // page fault
            sp = get_mfu(sp);
```

```
            frames[sp] = ref[i];
            count[sp] = 1;
            faults++;
            sp = (sp + 1) % n;
        }
        else
        {
            count[k]++;
        }

        for(j = 0; j < n; j++)
            mem[j][i] = frames[j];
    }
}

int main()
{
    int i, j;

    // Initialize
    for(i = 0; i < MAX; i++)
    {
        frames[i] = -1;
        count[i] = 0;
        for(j = 0; j < MAX; j++)
            mem[i][j] = -1;
    }

    faults = sp = 0;

    accept();
    mfu();
    disp();

    return 0;
}
```